PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS
Instituto de Ciências Exatas e de Informática

Teoria dos Grafos e Observabilidade: análise comparativa de arquiteturas *backend* sob estresse de viralização*

Gabriel Assunção Costa[1]
Matheus Henrique Resende Magalhães[2]
Bernardo Jeunon de Alencar[3]

Resumo

O resumo é redigido em parágrafo único. Deve apresentar, de forma sucinta, o contexto, o problema, a justificativa, o objetivo, o desenvolvimento, os resultados e a conclusão do documento. O texto deverá conter de 100 a 250 palavras.

Palavras-chave:

Abstract

The abstract must be written in a single paragraph and should briefly present the context, problem, justification, objective, development, results, and conclusion of the document. It should contain between 100 and 250 words.

Keywords:

*Artigo apresentado ao Instituto de Ciências Exatas e Informática da Pontifícia Universidade Católica de Minas Gerais, campus Betim, como pré-requisito parcial para obtenção do título de Bacharel em Sistemas de Informação.

[1]Aluno do curso de Sistemas de Informação.
[2]Aluno do curso de Sistemas de Informação.
[3]Orientador do trabalho.

1 INTRODUÇÃO

Redes sociais permitem que conteúdos se propaguem rapidamente entre usuários e produzam variações intensas no volume de interação. A disseminação decorre das características do conteúdo e das decisões sucessivas de compartilhá-lo, enquanto a posição do usuário na rede condiciona o alcance potencial de cada publicação. Quando um conteúdo alcança usuários com grande capacidade de disseminação, compartilhamentos sucessivos podem gerar picos de requisições, processamento e persistência (Liang, 2021; Sepehr; Beigy, 2022). A teoria dos grafos permite representar as relações entre usuários e identificar posições estruturalmente relevantes. Neste trabalho, a rede social será modelada como um grafo dirigido, e o grau de saída será adotado como medida operacional do alcance potencial dos iniciadores, sem pressupor equivalência com influência efetiva (Zareie; Sakellariou, 2023).

O problema consiste em investigar como diferentes arquiteturas *backend* respondem ao estresse gerado por cascatas de informação em redes sociais. A comparação considera uma arquitetura síncrona, baseada em PHP-FPM, Laravel e MySQL, com fluxo síncrono e persistência transacional linha a linha, e uma arquitetura orientada a eventos, baseada em PHP com Swoole/Hyperf, RabbitMQ e Neo4j, com processamento assíncrono, filas e ingestão em lotes por processadores de tarefas.

A relevância do estudo está em aproximar propriedades estruturais da rede das consequências observáveis na infraestrutura de software. Medidas como alcance, profundidade, largura e velocidade requerem controle conjunto na comparação de cascatas (Juul; Ugander, 2021). A avaliação conjunta de grafos, carga e observabilidade pode oferecer uma análise experimental mais precisa da degradação e da resiliência das arquiteturas, sem restringir a comparação ao tempo de resposta nem pressupor a superioridade de uma composição.

O objetivo geral é analisar comparativamente o comportamento de duas composições arquiteturais completas durante a simulação de cascatas de informação associadas à viralização. Para alcançá-lo, são definidos os seguintes objetivos específicos: i) modelar redes e cascatas com diferentes distribuições de conectividade; ii) implementar geradores de carga capazes de reproduzir picos controlados; iii) submeter as duas arquiteturas ao mesmo estímulo; iv) coletar métricas de recursos, latência, vazão, erros, filas e rastreamento distribuído; e v) comparar saturação, degradação e recuperação após o pico.

O restante deste artigo está organizado da seguinte forma: o referencial teórico define os conceitos de redes, cascatas, arquiteturas e observabilidade; a seção de trabalhos relacionados examina estudos próximos à comparação proposta; a metodologia descreve a classificação da pesquisa e o desenho experimental; os resultados apresentam os dados obtidos; e as considerações finais sintetizam as conclusões.

2 REFERENCIAL TEÓRICO

Esta seção reúne os conceitos necessários para relacionar a estrutura das redes sociais ao comportamento das duas composições arquiteturais sob carga.

2.1 Redes sociais e teoria dos grafos

Um grafo é uma estrutura formada por vértices e arestas: no domínio deste estudo, cada vértice representa um usuário e cada aresta dirigida representa uma relação pela qual um conteúdo pode alcançar outro usuário. O grau de saída contabiliza as conexões que partem do usuário. Redes que crescem por conexão preferencial podem concentrar muitas relações em poucos vértices, produzindo distribuições desiguais de conectividade (Piva; Ribeiro; Mata, 2021). Neste trabalho, o grau de saída será uma variável operacional de centralidade, e o alcance indicará quantos usuários podem ser atingidos a partir de um iniciador. Influência será entendida como a capacidade efetiva de induzir comportamentos, não como sinônimo automático de conectividade (Zareie; Sakellariou, 2023).

Uma cascata de informação é o subgrafo formado pelos compartilhamentos de um conteúdo e pelas exposições que os antecedem. Sua profundidade é a maior distância, em gerações, entre o iniciador e um participante; sua largura corresponde à quantidade de ocorrências em uma mesma geração. Cascatas de tamanhos semelhantes podem possuir estruturas e mecanismos de difusão diferentes (Sepehr; Beigy, 2022; Juul; Ugander, 2021).

2.2 Viralização e efeito manada

Viralização corresponde à disseminação rápida e ampla de um conteúdo por sucessivos compartilhamentos. O efeito manada descreve a tendência de uma pessoa adotar um comportamento após observar a adesão de outras. A posição topológica do iniciador condiciona o público que pode ser alcançado, mas métricas estruturais não explicam, isoladamente, a decisão de retransmitir (Liang, 2021). Tamanho, profundidade, largura e evolução temporal não devem ser interpretados como dimensões independentes (Juul; Ugander, 2021). O uso do termo efeito manada delimita o mecanismo social simulado que poderá produzir concentração de eventos em curto intervalo.

2.3 Arquitetura síncrona

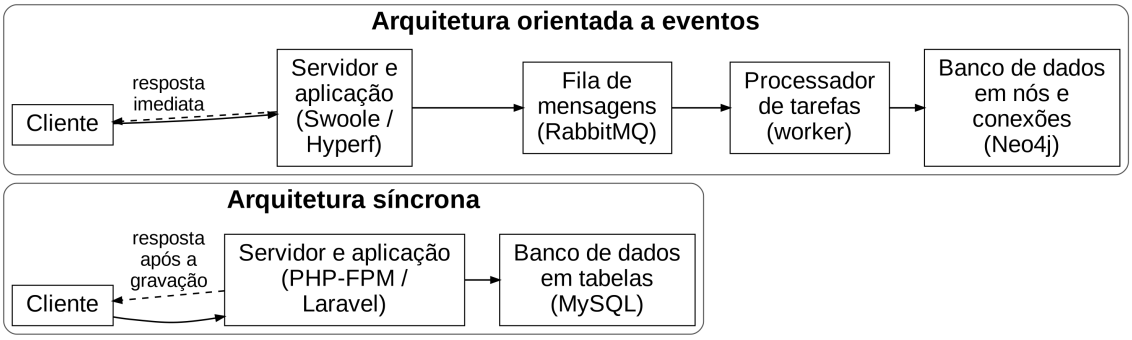
Na comunicação síncrona, o cliente envia uma solicitação e permanece em espera até que todo o processamento seja concluído e o resultado lhe seja devolvido (Karabey Aksakalli

et al., 2021). Na primeira composição avaliada neste estudo, a aplicação é construída com o *framework* Laravel e executada sobre o servidor PHP-FPM, responsável por receber as requisições e mantê-las em atendimento até que a resposta esteja pronta (PHP DOCUMENTATION GROUP, 2026). Ao receber cada solicitação, a aplicação aplica as regras da operação e registra o resultado no banco de dados MySQL, que organiza as informações em tabelas relacionadas entre si. O registro segue o mecanismo de transação: os dados só se tornam permanentes se a gravação for concluída por inteiro; caso contrário, ela é desfeita, o que impede o armazenamento de informações incompletas (ORACLE, 2026). Nessa composição, o tempo percebido pelo cliente corresponde, portanto, à soma do processamento da aplicação e da gravação necessária à resposta.

2.4 Arquitetura orientada a eventos

Em uma arquitetura orientada a eventos, o sistema confirma o recebimento da solicitação com rapidez e transfere o processamento mais demorado para execução posterior, a cargo de componentes independentes entre si. Na segunda composição avaliada neste estudo, a aplicação é construída com o *framework* Hyperf e executada sobre o servidor Swoole, que atende muitas solicitações simultâneas sem aguardar a conclusão de cada operação (SWOOLE, 2026). Cada solicitação relevante é convertida em mensagem e entregue ao RabbitMQ, serviço intermediário que a mantém em uma fila até distribuí-la aos processadores de tarefas (*workers*), programas que executam as tarefas e efetivam a gravação dos dados. A resposta ao cliente pode, assim, ser devolvida logo após o registro da mensagem, sem esperar todo o processamento subsequente (RABBITMQ, 2026). O contraste entre esse fluxo de atendimento e o descrito na seção anterior está ilustrado na Figura 1.

Figura 1 – Fluxo de atendimento das arquiteturas síncrona e orientada a eventos



Fonte: Elaborada pelo autor

A persistência definitiva é realizada no Neo4j, banco que, em vez de tabelas, armazena os dados como nós e conexões, representação próxima do domínio de redes sociais deste trabalho (NEO4J, 2026). Para dar conta de grande volume de mensagens em curto intervalo, os processadores de tarefas reúnem vários eventos em um mesmo lote antes de gravá-los, prática

denominada ingestão em lotes, que reduz o custo de cada gravação. Contudo, lotes excessivamente grandes elevam o consumo de memória e atualizações muito extensas precisam ser divididas em unidades menores (NEO4J, 2026).

2.5 Observabilidade sob carga

Observabilidade é a capacidade de inferir o estado interno de um sistema a partir dos sinais que ele produz. Métricas são séries numéricas agregadas; *logs* são registros discretos de eventos; e rastreamento distribuído acompanha uma operação pelas etapas e componentes que a processam (Tzanettis et al., 2022). O OpenTelemetry será empregado como estrutura independente de fornecedor para gerar e exportar esses sinais (OPEN TELEMETRY, 2026).

Sob carga, a latência é o tempo que o sistema leva para concluir uma operação, e a vazão é a quantidade de operações concluídas por unidade de tempo. Como as operações não demoram todas o mesmo tempo, a latência de cada cenário será resumida em percentis: o p50 corresponde ao limite que metade das operações não ultrapassa, enquanto o p95 e o p99 evidenciam os momentos mais críticos, pois indicam o limite superado apenas pelos 5

3 TRABALHOS RELACIONADOS

Abella, San Miguel e Ramasco (2023) analisaram o efeito do envelhecimento dos agentes na dinâmica temporal de cascatas. O estudo contribui ao distinguir as condições de ocorrência da cascata de sua velocidade de propagação. Diferentemente deste TCC, concentra-se na dinâmica do modelo social, e não na resposta de arquiteturas *backend*.

Cabane e Farias (2024) compararam empiricamente versões monolítica e orientada a eventos de uma mesma aplicação. A semelhança com este TCC é a comparação experimental de composições arquiteturais; a diferença está no estímulo, pois o presente estudo empregará cascatas derivadas de propriedades topológicas e também observará filas e rastros distribuídos.

Kotiranta, Junkkari e Nummenmaa (2022) compararam Neo4j, MySQL e MariaDB por meio de consultas com estruturas distintas. A semelhança está na presença de persistência relacional e orientada a grafos; a diferença é que o presente TCC avaliará ingestão durante cascatas de carga em arquiteturas completas.

Em síntese, os três estudos oferecem fundamentos para controlar a dinâmica temporal, comparar estilos arquiteturais e interpretar diferenças entre bancos. Não foi identificada uma avaliação que transforme propriedades topológicas de cascatas sociais em estímulo controlado para comparar uma composição síncrona e outra orientada a eventos.

4 METODOLOGIA

Esta seção descreve a classificação da pesquisa, as etapas planejadas, as ferramentas empregadas e o desenho experimental adotado para a comparação das arquiteturas.

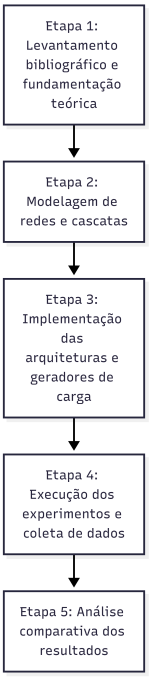
4.1 Classificação da pesquisa

A pesquisa é quantitativa, pois mensura variáveis numéricas como latência, vazão e utilização de recursos, submetendo-as a tratamento estatístico. Quanto à natureza, trata-se de uma pesquisa aplicada, pois visa gerar conhecimento utilizável na avaliação de sistemas backend submetidos a estresse. Quanto aos objetivos, é explicativa, pois busca identificar fatores que determinam o comportamento observado nas arquiteturas. Quanto aos procedimentos técnicos, o trabalho é experimental, pois submeterá duas composições arquiteturais a condições controladas de estímulo (Gil, 2022; Wazlawick, 2021).

4.2 Etapas da pesquisa

O desenvolvimento está estruturado em cinco etapas, ilustradas na Figura 2: i) levantamento bibliográfico e fundamentação teórica; ii) modelagem de redes sociais como grafos dirigidos e especificação de cascatas; iii) implementação das duas arquiteturas e desenvolvimento de geradores de carga; iv) execução dos experimentos e coleta de métricas; v) análise comparativa dos resultados e síntese das conclusões.

Figura 2 – Fluxograma das etapas da pesquisa



Fonte: Elaborada pelo autor

4.3 Ferramentas e tecnologias

O ambiente experimental é constituído pelas ferramentas apresentadas no Quadro 1. A escolha de cada componente justifica-se pela aderência aos objetivos do trabalho e pela disponibilidade de suporte a instrumentação.

Quadro 1 – Ferramentas e tecnologias do ambiente experimental

Categoria	Ferramenta	Função
Geração de carga	k6	Simulação de requisições HTTP
Backend assíncrono	PHP Swoole / Hyperf	Servidor orientado a eventos
Backend síncrono	PHP-FPM / Laravel	Servidor síncrono
Banco de dados (grafos)	Neo4j	Persistência orientada a grafos
Banco de dados (relacional)	MySQL	Persistência relacional
Message broker	RabbitMQ	Intermediação de mensagens
Coleta de métricas	Prometheus	Armazenamento de séries temporais
Visualização	Grafana	Dashboards e alertas
Rastreamento distribuído	OpenTelemetry	Geração e exportação de rastros
Conteinerização	Docker	Isolamento e orquestração de serviços

Fonte: Elaborado pelo autor

O k6 gerará cargas que simulam cascatas de compartilhamento. O PHP-FPM e o Laravel comporão a arquitetura síncrona, enquanto o Swoole e o Hyperf comporão a arquitetura orientada a eventos. O MySQL receberá a persistência da arquitetura síncrona, e o Neo4j, da arquitetura assíncrona. Prometheus, Grafana e OpenTelemetry constituirão a plataforma de observabilidade. O Docker será empregado para isolar e orquestrar os componentes.

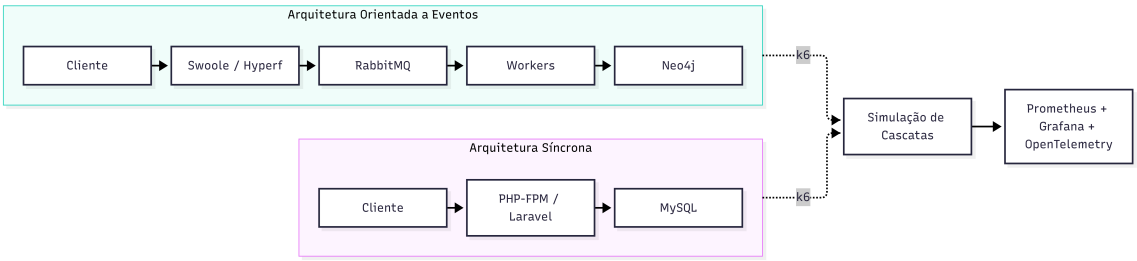
4.4 Desenho experimental

O desenho experimental está centrado na simulação de cascatas derivadas de propriedades topológicas de redes sociais. Redes serão modeladas como grafos dirigidos, e cascatas serão geradas a partir de nós com diferentes graus de saída. O gerador de carga, implementado com k6, produzirá requisições que representam compartilhamentos, respeitando a evolução temporal das cascatas.

As duas arquiteturas serão submetidas ao mesmo estímulo, definido por parâmetros como tamanho da rede, distribuição de grau, profundidade máxima da cascata e ritmo de propagação. Serão coletadas as medidas de desempenho definidas na Seção 2.5: a latência, resumida nos percentis p50, p95 e p99, e a vazão, além da taxa de erros, da utilização de CPU e de memória, do tamanho do *backlog* e do tempo de drenagem das filas. Rastros distribuídos permitirão correlacionar requisições, mensagens e operações de persistência.

A Figura 3 apresenta as duas composições arquiteturais. A arquitetura síncrona processa cada requisição de forma bloqueante, com persistência imediata. A arquitetura orientada a eventos desacopla a aceitação da requisição do processamento subsequente, utilizando filas e processadores de tarefas para ingestão em lotes.

Figura 3 – Diagrama comparativo das arquiteturas



Fonte: Elaborada pelo autor

Os experimentos serão organizados em cenários que variam a concentração de grau de saída dos iniciadores, o tamanho das cascatas e a intensidade do pico. Cada cenário será repetido múltiplas vezes para permitir análise estatística.

REFERÊNCIAS

ABELLA, D.; San Miguel, M.; RAMASCO, J. J. Aging in binary-state models: The threshold model for complex contagion. **Physical Review E**, v. 107, n. 2, p. 024101, 2023. Disponível em: https://doi.org/10.1103/PhysRevE.107.024101. Acesso em: 30 ago. 2026.

CABANE, H.; FARIAS, K. On the impact of event-driven architecture on performance: An exploratory study. **Future Generation Computer Systems**, v. 153, p. 52–69, 2024. Disponível em: https://doi.org/10.1016/j.future.2023.10.021. Acesso em: 30 ago. 2026.

GIL, A. C. **Como elaborar projetos de pesquisa**. 7. ed. Barueri: Atlas, 2022.

JUUL, J. L.; UGANDER, J. Comparing information diffusion mechanisms by matching on cascade size. **Proceedings of the National Academy of Sciences**, v. 118, n. 46, p. e2100786118, 2021. Disponível em: https://doi.org/10.1073/pnas.2100786118. Acesso em: 30 ago. 2026.

Karabey Aksakalli, I. et al. Deployment and communication patterns in microservice architectures: A systematic literature review. **Journal of Systems and Software**, v. 180, p. 111014, 2021. Disponível em: https://doi.org/10.1016/j.jss.2021.111014. Acesso em: 21 set. 2026.

KOTIRANTA, P.; JUNKKARI, M.; NUMMENMAA, J. Performance of graph and relational databases in complex queries. **Applied Sciences**, v. 12, n. 13, p. 6490, 2022. Disponível em: https://doi.org/10.3390/app12136490. Acesso em: 30 ago. 2026.

LIANG, H. Decreasing social contagion effects in diffusion cascades: Modeling message spreading on social media. **Telematics and Informatics**, v. 62, p. 101623, 2021. Disponível em: https://doi.org/10.1016/j.tele.2021.101623. Acesso em: 30 ago. 2026.

NEO4J. **Database Transactions**. 2026. Disponível em: https://neo4j.com/docs/operations-manual/current/database-internals/transaction-management/. Acesso em: 21 set. 2026.

OPEN TELEMETRY. **What is OpenTelemetry?** 2026. Disponível em: https://opentelemetry-.io/docs/what-is-opentelemetry/. Acesso em: 30 ago. 2026.

ORACLE. **MySQL 8.4 Reference Manual: START TRANSACTION, COMMIT, and ROLLBACK Statements**. 2026. Disponível em: https://dev.mysql.com/doc/refman/8.4/en-/commit.html. Acesso em: 21 set. 2026.

PHP DOCUMENTATION GROUP. **FastCGI Process Manager (FPM)**. 2026. Disponível em: https://www.php.net/manual/en/install.fpm.php. Acesso em: 21 set. 2026.

PIVA, G. G.; RIBEIRO, F. L.; MATA, A. S. Networks with growth and preferential attachment: Modelling and applications. **Journal of Complex Networks**, v. 9, n. 1, p. cnab008, 2021. Disponível em: https://doi.org/10.1093/comnet/cnab008. Acesso em: 30 ago. 2026.

RABBITMQ. **Consumer Acknowledgements and Publisher Confirms**. 2026. Disponível em: https://www.rabbitmq.com/docs/confirms. Acesso em: 21 set. 2026.

SEPEHR, A.; BEIGY, H. Structural virality estimation and maximization in diffusion networks. **Expert Systems with Applications**, v. 206, p. 117657, 2022. Disponível em: https://doi.org-/10.1016/j.eswa.2022.117657. Acesso em: 30 ago. 2026.

SWOOLE. **Swoole Documentation**. 2026. Disponível em: https://wiki.swoole.com/en/. Acesso em: 21 set. 2026.

TZANETTIS, I. et al. Data fusion of observability signals for assisting orchestration of distributed applications. **Sensors**, v. 22, n. 5, p. 2061, 2022. Disponível em: https://doi.org/10.3390-/s22052061. Acesso em: 30 ago. 2026.

WAZLAWICK, R. S. **Metodologia de pesquisa para ciência da computação**. 3. ed. Rio de Janeiro: LTC, 2021.

ZAREIE, A.; SAKELLARIOU, R. Influence maximization in social networks: A survey of behaviour-aware methods. **Social Network Analysis and Mining**, v. 13, n. 1, p. 78, 2023. Disponível em: https://doi.org/10.1007/s13278-023-01078-9. Acesso em: 30 ago. 2026.